

```
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2
```

```
23.8/23.8 MB 34.6 MB/s eta 0:00:00
232.6/232.6 kB 9.2 MB/s eta 0:00:00
```

```
import faiss
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM
from PyPDF2 import PdfReader
from google.colab import files
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

Using device: cpu

```
enc_name = "sentence-transformers/all-MiniLM-L6-v2"

enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)
enc_model = AutoModel.from_pretrained(enc_name).to(device)
enc_model.eval()
```

[Show hidden output](#)

```
llm_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

llm_tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm_tokenizer.pad_token = llm_tokenizer.eos_token

llm_model = AutoModelForCausalLM.from_pretrained(
    llm_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    device_map="auto"
)

llm_model.eval()
```

[Show hidden output](#)

```
uploaded = files.upload()
pdf_path = list(uploaded.keys())[0]
```

rag_sample...h_paper.pdf
rag_sample_research_paper.pdf(application/pdf) - 3667 bytes, last modified: 3/17/2026 - 100% done
Saving rag_sample_research_paper.pdf to rag_sample_research_paper.pdf

```
def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""

    for page in reader.pages:
        page_text = page.extract_text() or ""
        text += page_text + "\n"

    return text
```

```
def chunk_text(text, chunk_size=200, overlap=50):
    words = text.split()

    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap

    return chunks
```

```
def encode_chunks(chunks, batch_size=8):

    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(chunks), batch_size):

            batch = chunks[i:i+batch_size]

            tokens = enc_tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = enc_model(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)

            all_embeddings.append(
                embeddings.cpu().numpy().astype("float32")
            )

    return np.vstack(all_embeddings)
```

```
raw_text = extract_text_from_pdf(pdf_path)

doc_chunks = chunk_text(raw_text, chunk_size=200, overlap=50)

doc_embeddings = encode_chunks(doc_chunks)

faiss.normalize_L2(doc_embeddings)

dim = doc_embeddings.shape[1]

index = faiss.IndexFlatIP(dim)
index.add(doc_embeddings)

print("Total chunks indexed:", len(doc_chunks))
```

```
Total chunks indexed: 3
```

```
def retrieve_context(query, top_k=5):

    with torch.no_grad():
```

```

tokens = enc_tokenizer(
    [query],
    padding=True,
    truncation=True,
    return_tensors="pt"
).to(device)

outputs = enc_model(**tokens)

q_emb = outputs.last_hidden_state.mean(dim=1)

q_emb = q_emb.cpu().numpy().astype("float32")

faiss.normalize_L2(q_emb)

scores, indices = index.search(q_emb, top_k)

return [doc_chunks[i] for i in indices[0]]

```

```

def generate_rag_answer(question, top_k=5, max_new_tokens=256):

    contexts = retrieve_context(question, top_k)

    context_text = "\n\n".join(contexts)

    prompt = (
        "You are an assistant that answers ONLY using the provided context.\n\n"
        f"Context:\n{context_text}\n\n"
        f"Question: {question}\nAnswer:"
    )

    inputs = llm_tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True
    ).to(device)

    with torch.no_grad():

        output_ids = llm_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    output_text = llm_tokenizer.decode(
        output_ids[0],
        skip_special_tokens=True
    )

    return output_text.split("Answer: ")[-1].strip()

```

```

question = "What is the main contribution of this paper?"

answer = generate_rag_answer(question)

print("\nQuestion:", question)
print("\nAnswer:", answer)

```

Question: What is the main contribution of this paper?

Answer: This paper provides a simple overview of machine learning and neural networks.